SailNet: Multi-Task Frame Classification

Frame-Level Classification of Point of Sail, Tack and Boat Type from Sailing Video

Nicolò Nucci Chiara Mezzanzanica
Università degli Studi di Milano-Bicocca
{n.nucci,c.mezzanzanica2}@campus.unimib.it



Abstract — Reviewing sailing footage is slow, and the annotations that come out of it are still made by hand. We present SailNet, a multi-task classifier that labels every frame of a sailing video with its point of sail, its tack and the type of boat at the same time. To build it we put together a labelled dataset from 95 on-water sessions, extracted and annotated with a reproducible pipeline, and split it by session so that near-identical neighbouring frames never leak from training into test. On a shared EfficientNetV2-B2 backbone we compared four ways of connecting the three prediction heads, under identical data and augmentation. Letting the heads feed each other brought no clear benefit, whereas a lightweight spatial attention module on the tack head improved tack classification markedly, and a second, partial fine-tuning phase improved it further. On sessions never seen during training the tack is recognised with a macro accuracy of 0.85, a solid base for detecting tacks and gybes. The other two outputs are less mature: boat type generalises worse than validation suggested, and the no-go zone is not yet recognised, because only three of the six points of sail appear in our footage and the no-go zone is very scarce. The next step is therefore broader and more varied footage, followed by temporal models that turn per-frame labels into manoeuvre analysis.

Contents

1 Introduction 2
2 Dataset and Labelling Pipeline 2
2.1 Video Collection and Frame Extraction 2
2.2 Class Definitions 2
2.3 Dataset Statistics and Split 3
3 Multi-Head Architecture 4
3.1 Backbone and Shared Neck 4
3.2 The Three Prediction Heads 4
3.3 Focal Loss 4
3.4 Data Augmentation 4
3.5 Model A: Independent Heads 5
3.6 Model B: Full Cascade 5
3.7 Model C: Partial Cascade 5
3.8 Model D: Partial Cascade with Side Attention . 6
4 Training Protocol 6
4.1 Phase 1: Frozen Backbone 6
4.2 Phase 2: Progressive Unfreezing 6
5 Experimental Results 7
5.1 Evaluation Metrics 7
5.2 Phase 1 Architecture Comparison 7
5 Experimental Results 7
5.1 Evaluation Metrics 7
5.2 Phase 1 Architecture Comparison 7
5.3 Cascade Analysis: Model A vs. Model B 8
5.4 Model Selection for Phase 2 8
5.5 Phase 2 Fine-Tuning Results 8

5.6 Test Set Evaluation 8
6 Discussion 9
7 Conclusions and Future Work 9

1. Introduction

Video review is a large part of how sailors are coached. After a session, coach and athlete go through the footage to find the manoeuvres, check technique and compare tacks. At five frames per second, a ninety-minute session already produces about 27 000 frames, and nobody can annotate that consistently by hand on a regular basis.

An automatic classifier that answers three questions for every frame would remove most of that work: which point of sail the boat is on, which tack, and which type of boat it is. Together these labels describe the structure of a session, and tacks and gybes show up as transitions between labelled states. The rate and quality of those transitions could then be tracked from one session to the next. This paper covers only the first step, the per-frame classifier.

We treat it as a multi-output problem in which one convolutional network produces three predictions from a shared feature extractor. The design question we focus on is how the heads should exchange information. Boat type is easy to see and direction is hard, so it is tempting to let the direction head use the boat-type prediction as context. Whether this helps or only adds noise is not obvious, and the same doubt applies to port/starboard, which looks more like a spatial problem than something boat type could help with.

To look at this we train four architectures that change the information flow step by step: independent heads (Model A), a full cascade (Model B), a partial cascade (Model C), and the partial cascade with an attention block on the tack head (Model D). Our contributions are:

- a frame extraction and labelling pipeline for sailing video, with session-level train/validation/test splits to avoid leakage between neighbouring frames;
- a comparison of the four head designs under identical data and augmentation, in which the cascade connections gave no benefit and the attention block improved tack classification;
- a held-out evaluation that shows where the current system works (tack) and where the data, rather than the model, are the limit (point of sail, and to a lesser extent boat type).

2. Dataset and Labelling Pipeline

2.1 Video Collection and Frame Extraction

The raw material is a set of sailing videos recorded during on-water training sessions. Each video carries a single label triple (point of sail, tack, boat type) that is applied to every frame of the clip. The videos are listed in a spreadsheet with one row per file, and the extraction notebook checks that each label is valid and that the file exists.

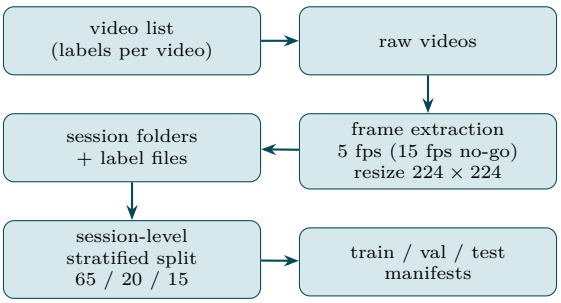


Figure 1: Labelling pipeline. The training notebook only reads the final manifests and the class information file.

Frames are extracted with `ffmpeg` (with OpenCV as a fallback) at 5 frames per second by default. The no-go zone is the exception. It covers a narrow angular window, roughly ±45° around the wind, and the heading barely changes from one frame to the next, so the class is both visually ambiguous and short on footage. For this class we extract at 15 frames per second. Every frame is then resized to 224×224 pixels with area interpolation, which is the input size of EfficientNetV2-B2.

Each video becomes a session folder named after the file, the boat type, the point of sail and the tack, with a label file that stores for every frame its index, the index in the source video and its timestamp in seconds. The overall flow is shown in Figure 1.

2.2 Class Definitions

Direction. Six points of sail, ordered by increasing angle from the wind: no-go zone, close-hauled, close reach, beam reach, broad reach and run (Figure 2). The order is not used in training; the task is plain multi-class classification. It still matters for interpreting errors, since neighbouring classes look alike.

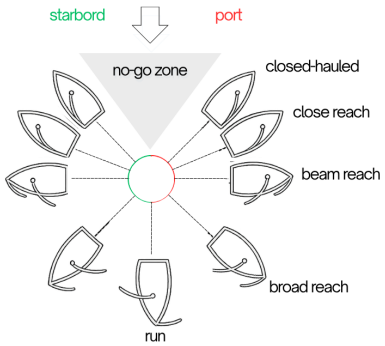


Figure 2: The six points of sail used as direction classes, relative to the wind direction, from closest to the wind (no-go zone) to deepest downwind (run).

Side. The tack, port or starboard, is defined by the side of the boat from which the wind arrives: on port tack the wind comes over the port side, on starboard tack over the starboard side. The wind itself is not visible in a frame, but the sail is trimmed on the opposite side of the hull, so its position tells the two tacks apart (Figure 3). This makes the task mainly a spatial one: the same sail shape means port or starboard depending on whether it appears on the left or on the right of the hull. The mapping also depends on where the camera is, since a

frame filmed from the front shows the mirror image of one filmed from behind.



Figure 3: Example of the two tack classes. The sail sits on opposite sides of the hull.

Boat type. ILCA or Optimist. The two classes differ in size and rig. The Optimist is a small single-handed dinghy for young sailors, with a blunt bow and a four-cornered sprit sail. The ILCA is a larger single-handed dinghy with a triangular sail and a more slender hull (Figure 4). Given these differences we expected this to be the easiest of the three tasks, which the validation results confirmed and the test set later contradicted (Section 5.6).

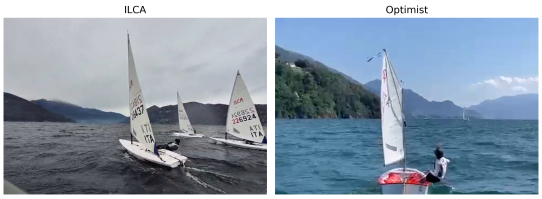


Figure 4: Example of the two boat classes, ILCA (left) and Optimist (right).

2.3 Dataset Statistics and Split

The final dataset has **8 775 frames** from **95 sessions**. The split into training, validation and test sets is 65/20/15 at the level of sessions, so that all frames of a video fall in the same partition. Consecutive frames of one video are almost identical in boat position, light and sea state, and a random split by frame would put near-duplicates in both training and test data. Test accuracy would then say very little about unseen footage. The split is stratified on the joint label (boat type, direction, side) and uses a fixed seed. The resulting sets contain 4 899 (train), 2 233 (validation) and 1 643 (test) frames, with the class counts in Figures 5 and 6.

Two features of the data shape most of what follows. The first is that the close reach, beam reach and broad reach classes have no frames at all. The direction head has six outputs, but it is trained and evaluated on three classes only. This reflects the footage we had available, not a problem in the pipeline. The second is the imbalance within the three classes that are present: close-hauled has 4 261 frames, run has 3 968 and the no-go zone only 546 (6.2%), even after sampling it three times more densely. In the training sessions the no-go zone amounts to about 21 seconds of real video.

Frame counts hide a second difficulty, which is how many independent videos stand behind each number. Table 1 lists sessions, frames and real duration for each class and partition. Durations are the number of frames divided by the sampling rate (5 fps, 15 fps for the no-go zone). All counts are taken before augmentation.

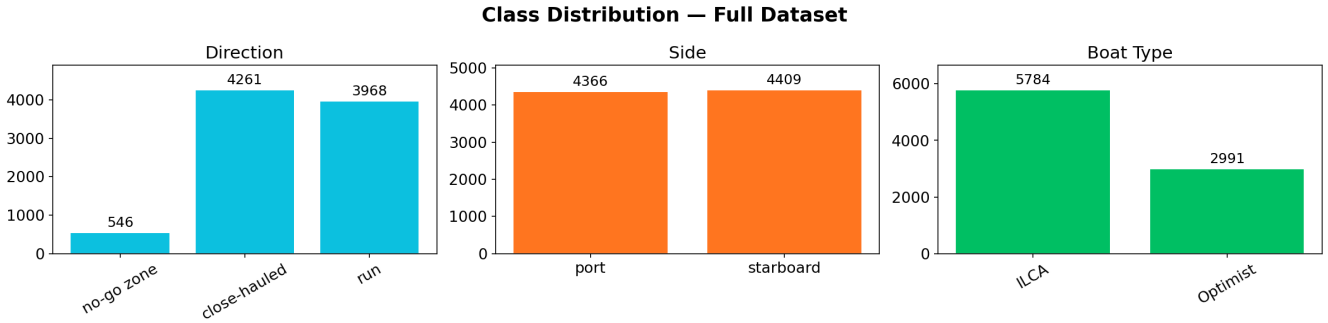


Figure 5: Frame counts per class over the full dataset. Only three of the six direction classes are present: close-hauled, run and no-go zone.

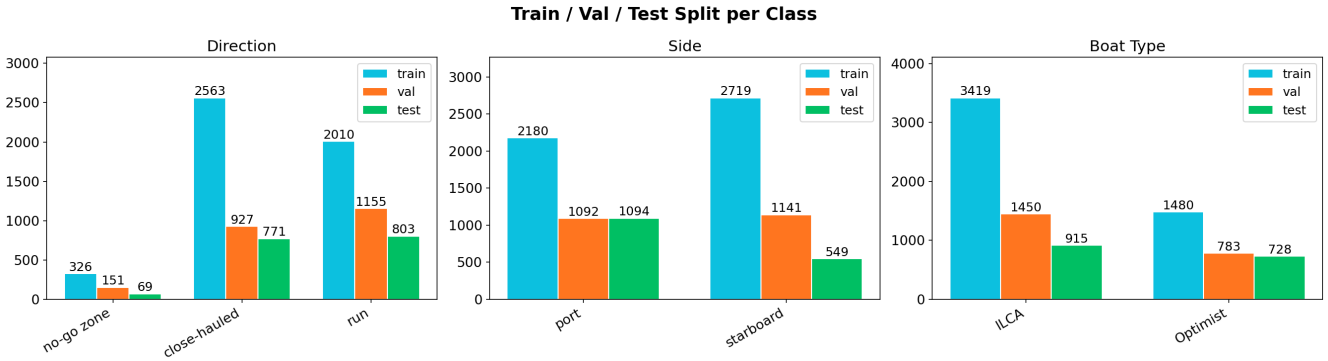


Figure 6: Class counts in the training, validation and test partitions. The no-go zone has 326, 151 and 69 frames respectively, and the test set contains twice as many port frames as starboard frames (1 094 against 549).

A no-go zone session is short, 25 frames on average (about 1.7 seconds), against 83 for close-hauled and 118 for run. The class is therefore thin in two ways at once, with few frames and few independent clips, and the second matters more because frames of one clip are nearly identical. The test set has only 3 no-go zone sessions, with 29, 29 and 11 frames, i.e. less than 5 seconds of footage in total.

Split	Class	Sessions	Frames	Seconds
Train	close-hauled	31	2 563	513
	run	17	2 010	402
	no-go zone	13	326	22
Val	close-hauled	9	927	185
	run	6	1 155	231
	no-go zone	4	151	10
Test	close-hauled	7	771	154
	run	5	803	161
	no-go zone	3	69	5

Table 1: Sessions, frames and real duration per direction class and partition, before augmentation.

3. Multi-Head Architecture

In multi-task learning a single model is trained on several related tasks and shares part of its representation between them [1]. Boat type, tack and point of sail all start from the same image, so one feature extractor can serve three heads instead of training three networks.

3.1 Backbone and Shared Neck

All four models use the same backbone and neck. The backbone is EfficientNetV2-B2 [2] with ImageNet weights and no classification top. Starting from pre-trained weights makes sense for a dataset of 8 775 frames, since edges, textures and shapes learned on natural images transfer to boats and sails. The B2 variant fits on a Colab GPU and gives a $7 \times 7 \times 1408$ feature map, which keeps enough spatial resolution for the attention block used in Model D. Inputs are normalised with the ImageNet mean and standard deviation.

The neck reduces the feature map to a single vector. A 1×1 convolution with 256 channels, batch normalisation and ReLU are followed by global average pooling and a dense layer of 512 units (batch normalisation, ReLU, dropout with $p = 0.3$). All weights use L2 regularisation with $\lambda = 10^{-4}$.

3.2 The Three Prediction Heads

Each model has three heads, one per attribute. All of them have the same internal structure: a dense layer of 256 units with batch normalisation, ReLU and dropout ($p = 0.15$), followed by a softmax layer. What changes from one head to the next is the number of outputs and the input they receive.

Boat head. Two outputs, ILCA and Optimist. It always reads the neck vector and nothing else, so it is the same in all four models. It is the easiest task, since the two boats differ clearly in hull and sail.

Side head. Two outputs, port and starboard. In Models A, B and C it reads the neck vector, with the addition of the boat logits in Model B. In Model D it does not use the neck at all and reads the attended $7 \times 7 \times 1408$ feature map instead (Section 3.8). This is the head that differs most between models, because the tack depends on where the sail sits relative to the hull, a cue that the pooled neck vector tends to lose.

Direction head. Six outputs, one per point of sail, of which only no-go zone, close-hauled and run have training data (Section 2). It reads the neck vector, and in Models B, C and D it also receives the logits of the boat head (and, in Model B, of the side head), concatenated to the neck vector. This is the hardest of the three tasks, because neighbouring points of sail differ only by a few degrees of heading relative to the wind.

Whenever a head receives the output of another one, it is the *logits* that are passed, through a `stop_gradient`, so that the loss of the downstream head cannot alter the upstream one. The four architectures differ only in these connections and in the side head of Model D, which is what Sections 3.5 to 3.8 describe.

3.3 Focal Loss

With one direction class much rarer than the others, plain cross-entropy lets the majority classes dominate the gradient. We therefore use a focal loss, following Lin et al. [3], which multiplies the cross-entropy by $(1 - p_t)^\gamma$, where p_t is the probability assigned to the correct class and $\gamma = 2$:

$$\mathcal{L}_{\text{focal}}(p_t) = -(1 - p_t)^\gamma \log(p_t). \tag{1}$$

Frames that are already classified with confidence contribute little, while hard frames and rare classes keep most of their weight. We add label smoothing with $\epsilon = 0.1$ [4] on the cross-entropy target. The focal factor is computed on the original hard labels, so that smoothing does not inflate it.

The three heads are combined in a weighted sum,

$$\mathcal{L} = 0.5\,\mathcal{L}_{\text{dir}} + 0.3\,\mathcal{L}_{\text{side}} + 0.2\,\mathcal{L}_{\text{boat}}. \tag{2}$$

These weights express how important we consider each task: direction is the main output and the hardest, tack matters for manoeuvre detection, and boat type is close to metadata. They were chosen in advance and not tuned.

3.4 Data Augmentation

We use three augmentation pipelines built with Albumentations [5]. Each training frame gives the orig-

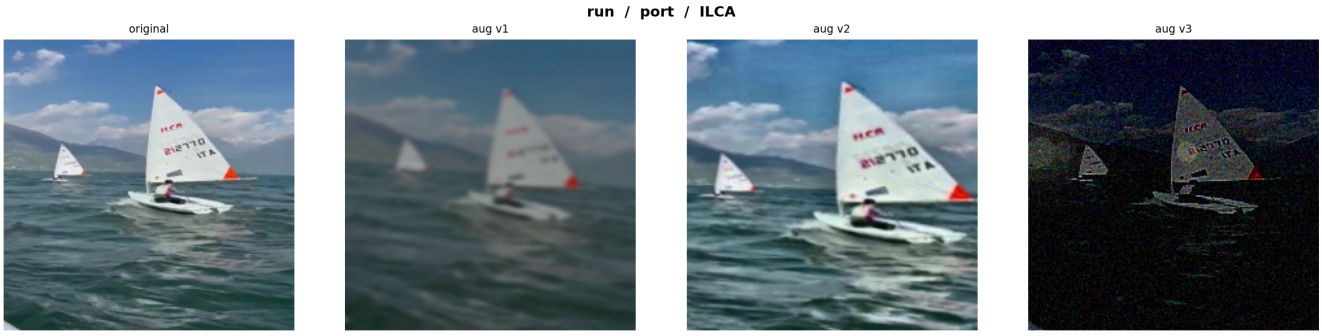


Figure 7: Augmentation example for a run, port-tack ILCA frame. From left to right: the original frame and the outputs of pipelines V1, V2 and V3. The random seed of each copy is derived from the image path, so the same pixels are produced in every training run.

inal plus three augmented copies, so one epoch has $4 \times 4\,899 = 19\,596$ training samples.

- V1: rotation up to 25°, colour jitter, fog and motion blur;
- V2: perspective warp, random resized crop, hue and saturation shift, CLAHE and random shadow;
- V3: affine transform, brightness and contrast change, sharpening, sun flare, Gaussian noise and coarse dropout.

The weather and lighting transforms (fog, random shadow, sun flare, brightness and contrast changes, hue and saturation shifts) were chosen with a specific aim. Our videos were recorded on a limited number of days and venues, so they cover only a narrow range of sky, light and sea conditions. A model trained on them alone could learn to rely on those conditions, for example on the colour of the water or the position of the sun, instead of on the boat and the sail. By simulating overcast, hazy, glaring and shaded scenes we want the classifier to stay reliable in weather that is not present in our footage. This is a design choice, not a result: we did not build a test set with different weather, so we cannot say how much robustness these transforms actually add. Synthetic fog and flare are also only an approximation of real conditions.

There is no horizontal flip, because mirroring a frame would swap port and starboard and make the tack label wrong.

Augmentation is applied on the fly. The seed of each augmented copy comes from a SHA-256 hash of the image path and the pipeline index, which means that all four models see exactly the same augmented pixels. Without this, a difference between two models could come from the augmentation instead of the architecture. Validation and test frames are only resized (Figure 7).

3.5 Model A: Independent Heads

In Model A every head reads the neck vector and nothing else (Figure 8). It is the reference point. A cascade can only be judged against an independent baseline that was trained in the same way.

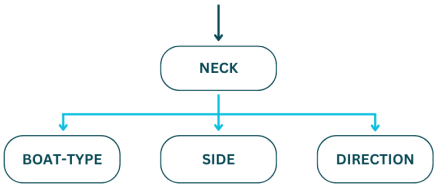


Figure 8: Model A. Three independent heads read the shared neck.

3.6 Model B: Full Cascade

Model B follows the idea that boat type limits the plausible tacks, and that boat type plus tack limit the plausible points of sail. The chain is boat → side → direction (Figure 9). The context is passed as logits, not probabilities, because a softmax squeezes the scores into $[0, 1]$ and loses how much more confident the model is in one class. A `stop_gradient` is applied on each link, so that the loss of a downstream head cannot change the upstream head.

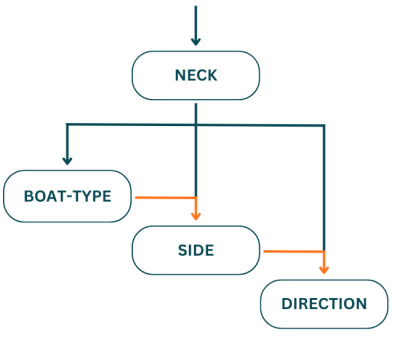


Figure 9: Model B. Boat logits feed the side head, and boat and side logits feed the direction head, all through `stop_gradient`.

3.7 Model C: Partial Cascade

Model C was designed after the first comparison between A and B (Section 5.3). It drops the boat → side and side → direction links and keeps boat → direction (Figure 10). It is also the control for Model D, since the only difference between the two is the side head.

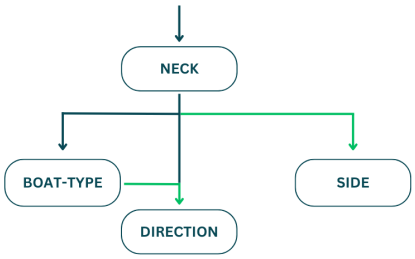


Figure 10: Model C. Boat logits feed only the direction head; the side head reads the neck as in Model A.

3.8 Model D: Partial Cascade with Side Attention

Model D has the cascade of Model C and a different side head. The reason is that global average pooling in the neck discards where things are in the image, while port versus starboard depends on which side of the hull the sail sits. A head that reads the pooled vector cannot see this.

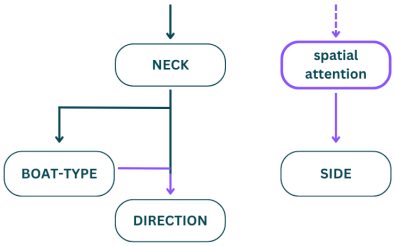


Figure 11: Model D. Same cascade as Model C, with a CBAM-lite attention block on the side head that works on the $7\times7\times1408$ feature map before pooling.

Why attention. The backbone output is a 7×7 grid of 1408-dimensional descriptors, each covering a patch of roughly 32×32 pixels of the input. Averaging this grid into a single vector gives the same result wherever a feature appears. For the tack, position is the information: the same sail shape means port or starboard depending on whether it is on the left or on the right of the hull. Attention mechanisms address this by letting the network decide, for each channel and each location, how much it should count [6].

CBAM-lite block. We use a reduced version of CBAM, with two steps applied in sequence to the feature map $F \in \mathbb{R}^{7\times7\times1408}$. *Channel attention* asks which feature maps matter for the tack. The global average-pooled and global max-pooled descriptors of F are passed through the same two-layer bottleneck ($1408 \to 352 \to 1408$ units), summed, and squashed by a sigmoid into one weight per channel:

$$M_c = \sigma\big(\mathrm{MLP}(\mathrm{AvgPool}(F)) + \mathrm{MLP}(\mathrm{MaxPool}(F))\big), \qquad F' = M_c \odot F \tag{3}$$

Spatial attention then asks where to look. The channel-refined map F' is reduced to a single channel by average and max over the channels, and a 7×7 convolution followed by a sigmoid produces one weight for each of the 49 positions:

$$M_s = \sigma\big(\mathrm{Conv}_{7\times7}([\,\mathrm{Avg}_c(F');\,\mathrm{Max}_c(F')\,])\big), \qquad F'' = M_s \odot F' \tag{4}$$

Since the kernel is as large as the grid, every output position can depend on the whole map, so in principle the block can learn a weighting that favours, for example, the left or the right side of the hull. The re-weighted map F'' is finally globally pooled and sent to the side head, which in this model does not read the neck vector. Model D has 11.0 M parameters (2.2 M trainable with a frozen backbone), against 9.7 M (0.9 M trainable) for A, B and C.

What we can and cannot claim. The motivation above is a design argument. What our experiments show is that D has higher side accuracy than C (0.816 against 0.690) with the same cascade. They do not show *why*. Two things change at once between C and D: the side head reads the unpooled map, and that map goes through attention. We did not train a variant that reads the unpooled map without CBAM, nor did we inspect the learned attention maps, so we cannot say how much of the gain comes from the attention itself.

4. Training Protocol

Training has two phases, with the backbone frozen in the first and partly unfrozen in the second.

4.1 Phase 1: Frozen Backbone

In Phase 1 all 347 backbone layers keep their ImageNet weights, and only the neck and the heads are trained. With less than 5 000 distinct training frames this keeps the number of free parameters small and the training stable. It also produces the checkpoints from which Phase 2 starts.

We use Adam with learning rate 10^{-3} and weight decay 10^{-4}, a batch size of 32 and at most 60 epochs. A `ReduceLROnPlateau` callback halves the learning rate after 2 epochs without improvement of the validation loss, and early stopping with a patience of 5 epochs restores the best weights. The four models stopped after 10 (A), 19 (B), 15 (C) and 12 (D) epochs.

Training ran on Google Colab, where sessions are interrupted often. The standard Keras callbacks reset their patience counters whenever `fit()` is called again, so we wrote small subclasses that save these counters, the learning rate and the best loss to a JSON file after every epoch and restore them on resume.

4.2 Phase 2: Progressive Unfreezing

Phase 2 starts from the best Phase 1 checkpoint of the selected model and unfreezes the last layers of the backbone. The early layers encode generic edges and textures that fine-tuning is unlikely to improve with this amount of data, while the last layers encode shapes and context, which are the ones affected by the change from everyday photographs to boats and water.

We start with the last 10% of the backbone unfrozen (34 of 347 layers). Each time the validation loss does not improve for 4 epochs, 1% more (3 layers) is unfrozen, up to a limit of 15% (52 layers). Releasing the backbone in steps seemed safer than unfreezing it all at once on a small dataset, although we did not compare the two options. Figure 12 shows how much of the backbone is trainable. Only the final number of unfrozen layers (46) was logged, not the epoch at which each step happened, so the figure shows the limits of the schedule and not the trajectory of that particular run.

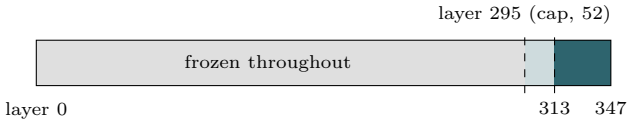


Figure 12: Backbone layers in Phase 2. Dark: unfrozen from the start (last 34 layers, 10%). Light: released step by step, up to the cap of 52 layers (15%). Model D ended with 46.

The learning rate is 10^{-3} at the start of Phase 2 and is halved after 3 epochs without improvement, with a floor of 10^{-5}. Batch-normalisation layers stay trainable. Early stopping is switched off for the first 5 epochs, because the validation loss often gets worse for a short time right after layers are unfrozen, and it uses a patience of 8 epochs afterwards. Phase 2 of Model D lasted 24 epochs and ended with 46 backbone layers unfrozen. Table 2 collects the settings of the two phases.

Setting	Phase 1	Phase 2
Trainable backbone layers	0 / 347	34 → 52 / 347
Optimiser	Adam	Adam
Initial learning rate	10^{-3}	10^{-3}
Weight decay	10^{-4}	10^{-4}
Batch size	32	32
Max. epochs	60	40
LR reduction	×0.5, pat. 2	×0.5, pat. 3
Minimum LR	–	10^{-5}
Early stop patience	5	8 (off first 5)
Unfreezing step	–	+3 layers / 4 stalls
Batch norm in backbone	frozen	trainable
Epochs run (Model D)	12	24

Table 2: Training settings of the two phases.

5. Experimental Results

5.1 Evaluation Metrics

We report accuracy and *macro accuracy*, the mean of the per-class recalls. Because of the class imbalance, standard accuracy is dominated by the large classes: a direction head that always answers close-hauled would score above 0.45 on the test set (771 of 1 643 frames) while ignoring two classes. Architecture comparison and model selection use the validation set. The test set was evaluated once, at the end.

5.2 Phase 1 Architecture Comparison

Table 3 gives the validation results for the four models after Phase 1, and Figure 13 the training curves.

Model	Boat	Side	Direction
Accuracy			
A (independent)	0.999	0.716	**0.936**
B (full cascade)	0.999	0.715	0.914
C (partial cascade)	**1.000**	0.690	0.893
D (partial + attn.)	0.999	**0.816**	0.927
Macro accuracy			
A (independent)	0.999	0.703	0.722
B (full cascade)	**1.000**	0.714	**0.778**
C (partial cascade)	**1.000**	0.686	0.655
D (partial + attn.)	0.999	**0.816**	0.706

Table 3: Validation results after Phase 1. Bold marks the best value in each column.

Boat type is close to perfect for every model, so it says little about the architectures. The direction results are mixed. Model A has the highest accuracy, and Model B has the highest macro accuracy (0.778), mostly because its no-go zone F1 is higher (0.331 against 0.268 for A, 0.212 for D and 0.075 for C). That class has only 151 validation frames, so these differences are fragile. The large differences are in the tack. Model D reaches 0.816 against 0.703 for A and 0.686 for C, and it is the only model whose port and starboard F1 are similar (0.806 and 0.825).

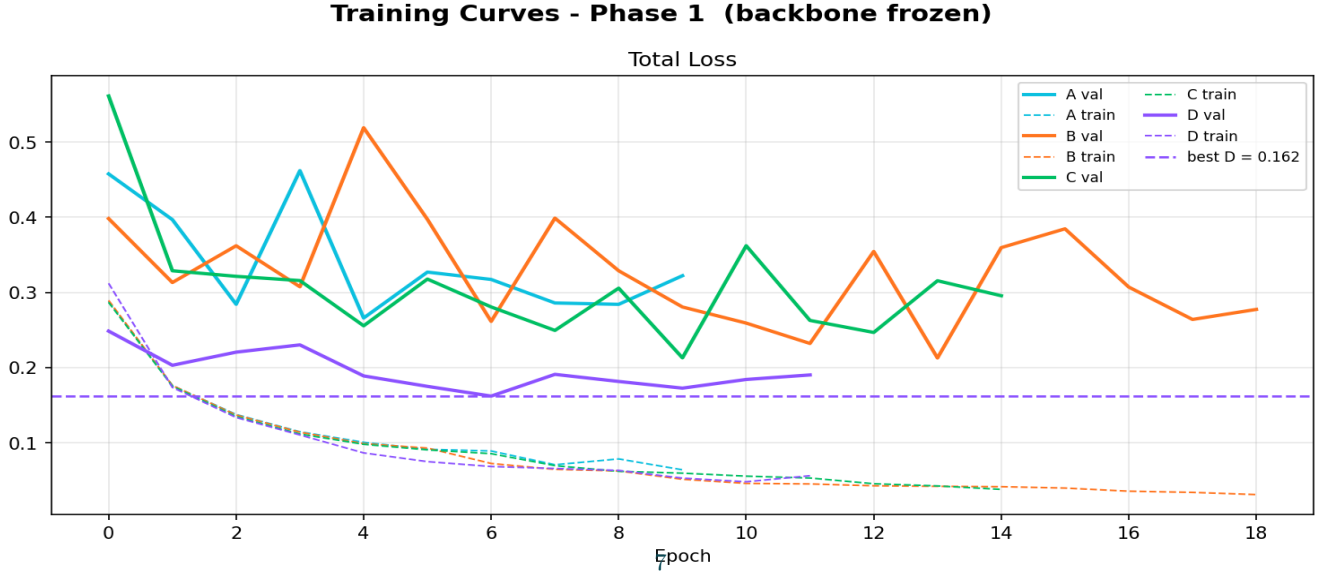


Figure 13: Phase 1 training (dashed) and validation (solid) curves for the four models. The validation curves are very noisy, in particular for side accuracy.

5.3 Cascade Analysis: Model A vs. Model B

Model B was meant to improve on A, and it did not. Side accuracy is the same (0.715 against 0.716), direction accuracy drops by 2.3 points, and port recall falls from 0.73 to 0.58. Our reading of this, which we have not tested, is that the boat-type logits contain nothing about where the sail is, so they add noise to a decision that depends on exactly that, and that the weaker side logits then mislead the direction head.

Model C does not rescue the idea either. It keeps only the boat → direction link and is the weakest model on direction (accuracy 0.893, macro accuracy 0.655, no-go zone F1 0.075). So our data give no sign that boat-type context helps the direction head. The gain of Model D comes from the attention block: with the same cascade as C, its side accuracy is 12.6 points higher.

5.4 Model Selection for Phase 2

We chose Model D for Phase 2, and the reason is the tack. Its side accuracy is 0.816, against 0.716 for A, 0.715 for B and 0.690 for C, and it is the only model whose port and starboard F1 scores are balanced (0.806 and 0.825). The other three are clearly weaker on one of the two sides. The price is small. On direction, D reaches an accuracy of 0.927, which is 0.9 points below A (0.936) but above B (0.914) and C (0.893). Its macro accuracy is also lower than that of A and B (0.706 against 0.722 and 0.778), mostly because of the no-go zone, which has only 151 validation frames. We do not read these gaps as a real loss on direction. They are smaller than the noise visible in Figure 13, and each model was trained only once.

The choice was made by hand and not from a single metric. A manoeuvre can only be detected if tack and direction are both right at the same time, so a model that is slightly better on direction but unreliable on the tack is of little use for our purpose. We therefore accepted a small and uncertain difference on direction in exchange for a large and consistent gain on side. We do not claim that D is better on direction than the other models.

5.5 Phase 2 Fine-Tuning Results

Table 4 compares Model D before and after fine-tuning on the validation set, and Figure 14 shows the curves of both phases.

Metric	Phase 1	Phase 2	Δ
Boat accuracy	0.999	0.993	−0.007
Side accuracy	0.816	0.865	+0.049
Direction accuracy	0.927	0.916	−0.011
Direction macro acc.	0.706	0.741	+0.035
Recall, no-go zone	0.184	0.322	+0.138
Recall, close-hauled	0.933	0.901	−0.033

Table 4: Model D on the validation set, Phase 1 and Phase 2.

Fine-tuning improves the tack by 4.9 points (port recall from 0.817 to 0.845, starboard from 0.816 to 0.883) and lifts the no-go zone recall from 0.18 to 0.32. The price is a lower close-hauled recall, since 8% of the close-hauled frames are now predicted as no-go zone (5% before), and a small loss on boat type. One detail should be kept in mind. The best Phase 2 validation loss (0.167) is slightly above the best Phase 1 value (0.162), because the Phase 2 checkpoint criterion starts again from scratch. We kept the Phase 2 model for its per-head accuracies, not for a lower loss.



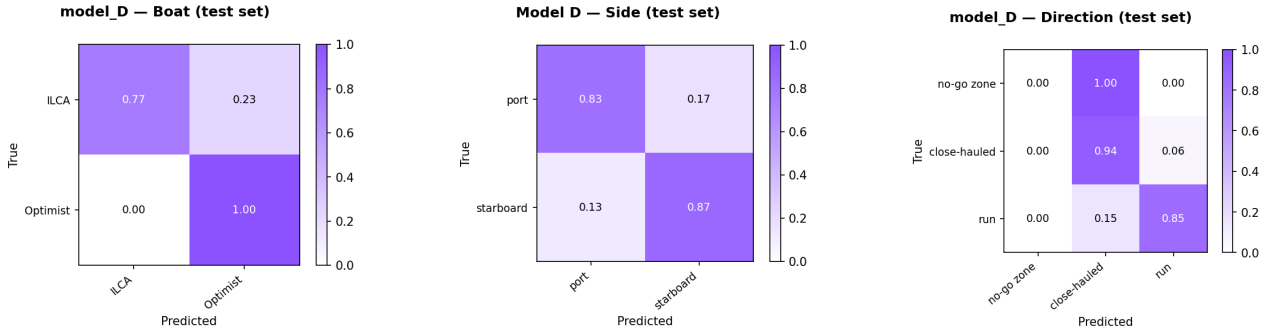
Figure 14: Model D, Phase 1 (blue) followed by Phase 2 (red).

5.6 Test Set Evaluation

The test set was not used for training, validation or model selection. Model D after Phase 2 was evaluated on it once (Table 5 and Figure 15). The tack holds up on unseen sessions: port and starboard recall are 0.83 and 0.87, about one point below the validation result, even though the test set has twice as many port frames as starboard frames. The boat type is less good than it looked. Macro accuracy is 0.884 against 0.99 on validation.

Head	Macro acc.	Recall per class
Boat type	0.884	ILCA 0.77, Optimist 1.00
Side	0.852	port 0.83, starboard 0.87
Direction	0.594	no-go 0.00, close-hauled 0.94, run 0.85

Table 5: Test results for Model D after Phase 2. Per-class recall is read from the confusion matrices.



(a) Boat type (b) Side (c) Direction

Figure 15: Row-normalised confusion matrices on the test set (Model D, Phase 2).

All Optimist frames are correct, but 23% of the ILCA frames are labelled Optimist. We do not know why. A likely explanation is that some test ILCA sessions differ from the training ones in background, camera distance or sail appearance, but this would have to be checked session by session, and we have not done it.

Direction is the weakest head. Standard accuracy is 0.853, and the macro accuracy is 0.594 because the model never predicts the no-go zone on the test set: all 69 no-go frames are labelled close-hauled. The partial recovery of this class on the validation set (recall 0.32) did not transfer.

The most likely reason is how little of this class the test split contains. It holds 69 frames from 3 sessions (Table 1), so the recall of 0.00 rests on three clips, and one or two of them being hard would be enough to produce it. We want to be careful about how far this explanation goes. Having few frames makes the estimate noisy, but it does not by itself say why the model fails: validation has only 4 sessions and still reaches 0.32, and the training data hold just 13. A model that has seen 13 short clips could also simply fail to generalise to new boats or angles. Telling the two possibilities apart needs recall per session, which we did not compute, so we present the small test split as the probable cause and not as an established one.

6. Discussion

Of the three tasks, the tack is the one we would call solved enough to build on. The attention block made the difference. Models A, B and C all had a weak port recall, which fits the idea that a pooled vector carries little information about where the sail is, and letting the head look at the feature map before pooling fixed most of it. We have not tested this explanation directly, for example by visualising the attention maps.

For direction, the problem is mainly the data. The model has seen three of the six points of sail, and the rarest of these, the no-go zone, is made of a handful of short clips (13 sessions, about 22 seconds in the training set). The test figure of 0.00 is based on only 3 sessions, so it is a weak measurement as well as a poor result. Another architecture is unlikely to fix a class with so little material, whereas more no-go zone footage from more boats would also give a test estimate that can be trusted. The cascade results point the same way. We expected boat context to help direction and found no sign of it.

The results also have limits that should be stated plainly.

- The effective sample size is the number of sessions, which is 95 in total. The test set holds about 15% of them, so a few clips can move the scores a lot. This is visible in the validation curves, and in the gap between validation and test for boat type (0.99 against 0.88) and direction (0.74 against 0.59).
- Every model was trained once, with one seed, and we report no confidence intervals. The direction heads of C and D have the same design and still differ by 3.4 points of accuracy, which suggests that run-to-run variation is of the same size as several of the differences we discuss. The conclusions on the cascades in particular are tentative.
- Each video has a single label, so a clip that contains a turn or part of a manoeuvre has frames whose label does not describe them.
- The weights of the loss, the dropout rate, γ and the unfreezing schedule were fixed in advance and not tuned.

7. Conclusions and Future Work

We built a pipeline from raw sailing video to a session-split frame dataset and trained a multi-head EfficientNetV2-B2 classifier for point of sail, tack and boat type. On the test set the final model reaches a macro accuracy of 0.852 for tack, 0.884 for boat type and 0.594 for point of sail. Among the four head designs we compared, the spatial attention block on the tack head was the only change that clearly helped, and the cascade connections did not. The tack result is good enough to build on. The boat type needs a closer look at the test sessions, and the point of sail needs more data before it can be judged as a modelling problem.

Two steps seem the most useful for the future.

Data. Footage for close reach, beam reach and broad reach, together with more and more varied no-go zone footage from different boats, venues and conditions, is the change most likely to improve direction. We would also repeat the training with several seeds and with

session-level cross-validation, so that differences between models can be reported with an uncertainty.

Temporal modelling. The classifier labels single frames, but detecting a tack or a gybe means following the labels over time, finding where a transition starts and how many frames it lasts, and eventually scoring how well it was done. A sequence model, such as a bidirectional LSTM or a Transformer over the per-frame outputs, is the natural next step. Once the manoeuvres are segmented, session-level numbers such as the number of tacks, the duration of a transition or the balance between port and starboard follow directly. Other boat classes, for example 470 or 49er, could be added with new labelled data and no change to the architecture.

References

[1] S. Ruder, "An overview of multi-task learning in deep neural networks," *arXiv preprint arXiv:1706.05098*, 2017.

[2] M. Tan and Q. V. Le, "EfficientNetV2: Smaller models and faster training," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021.

[3] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2999–3007, 2017.

[4] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[5] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, "Albumentations: Fast and flexible image augmentations," *Information*, vol. 11, no. 2, p. 125, 2020.

[6] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 3–19, 2018.

Use of AI Tools Large language models were used as writing tools during this project: to help write and debug the code of the notebooks. The research questions, the experimental design, the choice of architectures and hyper-parameters, the interpretation of the results and the conclusions are the authors' own, and the authors checked the reported numbers against the experiments.